

Power BI to Looker

Semantic Model Inventory & Migration Showcase

A detailed walkthrough of how every PBIX semantic object was captured, why an agentic architecture is required, what remains blocked (especially Power Query M dependencies), and how to unblock KPIs without dbt and without migrating reports/dashboards.

Case study: Microsoft / obviEnce Human Resources Sample PBIX
Scope: Semantic model only | Warehouse SQL (no dbt) | LookML drafts
Completeness gate: PASSED

Audience deliverable - methodology + evidence from a real PBIX extraction

1. Executive summary

This engagement demonstrated an inventory-first Power BI to Looker migration method. Instead of jumping straight to LookML, we decompressed the PBIX DataModel, extracted every semantic-model object, tagged each object with a clear next action, and documented blockers as explicit dependency chains (M code -> table -> KPI).

Reports and dashboards were intentionally excluded. The goal was a trustworthy semantic inventory and an honest statement of what Looker can and cannot do until warehouse tables and complex DAX patterns are implemented.

Key result

Completeness gate PASSED: 15 tables, 87 columns, 30 measures, 43 calculated columns, 6 calculated tables, 8 relationships, 9 Power Query scripts (verbatim .m files), 29 empty TM categories explicitly listed, 237 action-matrix rows.

2. Why agentic architecture is needed

A PBIX file is a compressed package. The analytical brain sits in a binary Analysis Services backup (often XPress9-compressed). Inside that model live multiple object families that do not convert 1:1 to LookML.

What a single-pass converter typically misses

- Power Query M SQL unions and seed tables (data cannot appear in Looker until rebuilt)
- Calculated columns that KPIs depend on (e.g. isNewHire -> New Hires)
- Time intelligence DAX (SAMEPERIODLASTYEAR) stubbed as NULL without disclosure
- Hidden auto date tables silently dropped (capture incomplete)
- Empty categories (RLS, KPIs) omitted instead of confirmed NONE_IN_SOURCE

Specialist agent roles

We used five specialist extractor roles plus a merger. Each role has one output contract. The merger fails the run unless completeness gates pass.

Agent role	Captures	Output
Schema	Tables, columns, types, stats	01_tables_columns.json
DAX	Measures, calc columns/tables (full text)	02_dax_objects.json
Relationships	M:1 joins, cross-filter, active	03_relationships.json
M / Power Query	Verbatim M + embedded SQL + tags	04_*.json + 04_m_raw/*.m
TM extras	Partitions, hierarchies, RLS, etc.	05_tmschema_extras.json
Merger + Gate	Inventory + action tags + pass/fail	OBJECT_INVENTORY + MATRIX

```
PBIX
-> Agent Schema
-> Agent DAX
-> Agent Relationships
-> Agent M / Power Query
-> Agent TM Extras
  \-> Merger + Completeness Gate
    -> OBJECT_INVENTORY.md
    -> ACTION_MATRIX.csv
    -> COMPLETENESS_GATE.json
```

Why this matters for audiences / auditors

Agentic separation creates evidence: every object class has a file, every empty class is listed, and LookML is not declared complete until the gate passes. In this project, roles ran inside one orchestrator script with identical contracts (logical multi-agent; single process for consistency).

3. End-to-end journey (what we did)

Step A - Locate and open the PBIX

Source: Human Resources Sample PBIX (obviEne / Microsoft sample). PBIX is a ZIP; DataModel uses XPress9 compression. We used pbixray on Python 3.12 to decompress and read the tabular model and mashup.

Step B - Extract inventory (Phase 1)

Ran capture_pbix_inventory.py to produce inventory artifacts and enforce the gate. Business tables and internal auto date tables were both captured.

Step C - Action matrix

Every object received exactly one primary action tag (LookML view/measure/join, warehouse SQL/seed, skip internal, or none-in-source). This becomes the backlog for conversion work.

Step D - Draft LookML (not final KPI parity)

Generated views/ and models/human_resources.model.lkml for the 9 business tables and 8 joins. Complex DAX left as TODO stubs with original expressions referenced.

Step E - Blockers document (honest gaps)

Wrote BLOCKERS_AND_DEPENDENCIES.md so every blocked KPI names its M-file or DAX dependency. No silent gaps.

4. Inventory evidence - tables

Business tables created via Power Query (M)

Table	Role	Cols	Origin
Employee	Fact (monthly snapshot)	16	M + SQL Server union
Date	Calendar dimension	12	M + SQL HR.Date
BU	Business unit / region	4	M + SQL
AgeGroup	Age bands	2	M embedded seed
Ethnicity	Ethnicity lookup	2	M embedded seed
FP	Full / Part time	2	M + SQL
Gender	Gender lookup	3	M embedded seed
PayType	Hourly / Salaried	2	M + SQL
SeparationReason	Vol / Invol	2	M + SQL

Internal auto date tables - captured, not ignored

Power BI auto-generated 6 LocalDateTable_* / DataTableTemplate_* tables for time intelligence. We captured: all columns, Calendar(...) DAX, and 36 calculated columns (Year, MonthNo, Month, Quarter, Day). Tagged SKIP_PBI_INTERNAL for default Looker migration - meaning optional to implement as LookML views, NOT meaning skipped in inventory.

Audience clarification

Capture everything first. Decide migration priority second. Internal tables prove the inventory is complete even when Looker will use dimension_group on Date instead.

5. Inventory evidence - DAX, relationships, M

Relationships (8) - all M:1

Employee.date -> Date.Date; Employee.BU -> BU.BU; Employee.AgeGroupID -> AgeGroup; Employee.EthnicGroup -> Ethnicity; Employee.FP -> FP; Employee.Gender -> Gender.ID; Employee.PayTypeID -> PayType; Employee.TermReason -> SeparationReason. Mapped to Looker left_outer + many_to_one joins.

Measures (30) - examples

Measure	Pattern	Action tag
New Hires	SUM(isNewHire)	LOOKML_MEASURE
Actives / Seps	Filtered counts	LOOKML_MEASURE
New Hires SPLY	SAMEPERIODLASTYEAR	LOOKML_TODO_COMPLEX
EmpCount	MAX(PeriodNumber) filter	LOOKML_TODO_COMPLEX
TO % Norm	ALL(Gender/Ethnicity)	LOOKML_TODO_COMPLEX
BadHire%ofActives	DIVIDE	LOOKML_MEASURE

Power Query M - verbatim capture

All 9 queries written to inventory/04_m_raw/*.m. Employee.m is the critical blocker: SQL Server query with UNION ALL of actives and separations, date shift +1 year, gender remap, EmplID%2 filter, month-grain snapshots.

6. When something is blocked - how we document it

Blocked does not mean forgotten. For every blocker we record: status, why, dependency artifact (.m / DAX), KPIs unlocked only after fix, and how to achieve without dbt.

Worked example - New Hires YoY %

```

BLOCKED: New Hires YoY % Change
+-- needs: New Hires YoY Var
    +-- needs: New Hires SPLY      <- LookML (SAMEPERIODLASTYEAR)
        +-- needs: New Hires      <- SUM(isNewHire)
            +-- needs: isNewHire  <- DAX calculated column
                +-- needs: hr.employee from Employee.m
                    <- M CODE DEPENDENCY (warehouse/PDT)

```

Blocker map (summary)

Blocker	Type	Impact
Employee.m not in warehouse	M dependency	Almost all KPIs
Dim M/SQL missing	M dependency	Region/time/pay slices
Seed dims missing	M dependency	Age/Gender/Ethnicity
Calc columns	DAX->SQL/LookML	New Hires, Bad Hires, Tenure
SPLY / YoY chain	LookML complex	All SPLY & YoY KPIs
EmpCount max period	LookML complex	EmpCount (+ Actives parity)
TO % Norm	LookML complex	TO % Norm / Var
KPI parity not run	Validation	Confidence, not build

Suggestion for delivery teams: never say 'not supported in Looker' alone. Say 'blocked on Employee.m warehouse load'

or 'blocked on SAMEPERIODLASTYEAR pattern' and list the KPI names above that blocker.

7. Conversion honesty - are we at 70-80%?

Layer	Progress	Meaning
Object capture / inventory	~95-100%	Gate passed
Dim/fact + join design	~80%	LookML drafts exist
Simple measures	~70%	Need real tables behind them
Complex measures	~15-25%	Stubs / SPLY chain
Warehouse tables (no dbt)	~0-10%	M extracted, not loaded
KPI parity vs Power BI	~0%	Not validated yet

Interpretation: ~70-80% of understanding/mapping is done. Runnable Looker replacement is lower until warehouse tables exist. After warehouse + simple measures, usable conversion can approach ~70%; 80%+ true parity needs complex DAX and KPI checks.

8. Recommended next steps (no dbt, no dashboards)

- Build warehouse views/tables (or Looker PDTs) from 04_m_raw/*.m
- Materialize or Keep LookML SQL for isNewHire, BadHires, tenure, AgeGroupID, Region
- Wire LookML sql_table_name / connection to those tables
- Implement SPLY, EmpCount, TO % Norm; then YoY measures unlock
- Run KPI parity checklist against Power BI for 10-15 core metrics
- Optionally generate LookML for internal date tables if product requires them

9. Artifact index for the showcase folder

Path	Purpose
inventory/01_tables_columns.json	Full schema capture
inventory/02_dax_objects.json	Full DAX text
inventory/03_relationships.json	Joins
inventory/04_power_query_m.json	M metadata
inventory/04_m_raw/*.m	Verbatim Power Query
inventory/05_tmschema_extras.json	TM extras + empties
inventory/OBJECT_INVENTORY.md	Human-readable inventory
inventory/ACTION_MATRIX.csv	Object -> action backlog
inventory/COMPLETENESS_GATE.json	Pass/fail evidence
views/ + models/	Draft LookML
BLOCKERS_AND_DEPENDENCIES.md	M->KPI blocker chains
MIGRATION_SUMMARY.md	Earlier migration notes
capture_pbix_inventory.py	Repeatable extractor

10. Closing message for the audience

Successful PBIX to Looker migration is not 'generate LookML and hope.' It is an evidence-based pipeline: specialist extraction, completeness gates, action tags, and explicit blocker chains from Power Query M up to business KPIs.

If something cannot be converted yet, we say so clearly: here is the M dependency, here are the KPIs calculated from it,

and here is how to achieve it in the warehouse or LookML - without hiding gaps behind a green 'migrated' status.

One-line thesis

Capture everything. Tag everything. Block with names. Then build LookML.